

CONSILIUM

AI Decision Copilot for Veterinary Clinics

Operational Deployment Plan

Event	UMHackathon 2026 — Final Round
Live Build	https://consilium-tau.vercel.app
Repository	github.com/Shawnchee/DA-Homies
Document Status	Production Ready

Contents

1. Project Overview	3
1.1 Clinical Loop	3
1.2 Tech Stack	3
2. Deployment Objectives	4
3. Deployment Architecture	5
3.1 Production Topology	5
3.2 Request Flow — Owner Message to Escalation Card	5
3.3 Failure Fallback	5
4. Environment Configuration	7
4.1 Required Environment Variables (Production)	7
4.2 Per-Feature Model Overrides	8
4.3 Secret Management	8
5. CI/CD and Deployment Workflow	9
5.1 Branch Strategy	9
5.2 End-to-End Deploy Flow	9
5.3 Vercel Ignored-Build-Step	9
5.4 AI Eval Pipeline (ai-eval.yml)	9
5.5 Database Migrations (db-migrate.yml)	10
6. Step-by-Step Deployment Process	11
Step 1 — Deploy Next.js to Vercel	11
Step 2 — Provision Supabase	11
Step 3 — Configure the Telegram Bot	11
Step 4 — Deploy the Python LangGraph Sidecar (Optional)	11
Step 5 — Production Validation Checklist	12
Step 6 — Schedulers	12
Clinic Brain Sync (.github/workflows/clinic-brain-sync.yml)	12
7. Monitoring and Error Handling	13
7.1 Where Logs Live	13
7.2 Failure Handling Already in Code	13
7.3 Current Observability Gaps	13
8. Rollback Plan	14
8.1 When to Roll Back	14
8.2 Rollback Steps (Target: Under 2 Minutes)	14
8.3 What Rollback Cannot Fix	14
9. Risks and Mitigation	15
10. Future Improvements	16
Appendix — Quick Reference	17

1. Project Overview

Consilium is a single-tenant web app + Telegram bot that runs the full clinical loop for a solo vet clinic. The plan deliberately stays at the level a small team can actually run — no Kubernetes, no Terraform, no aspirational infrastructure.

1.1 Clinical Loop

- Pre-consult — Claude Haiku 4.5 produces a 5-line patient brief on the dashboard.
- During consult — Claude Sonnet 4.6 captures voice (Deepgram nova-3) and photos (vision) and emits structured SOAP / Rx / billing / todos.
- Post-discharge — owners chat with @consilium_vet_bot on Telegram; Sonnet 4.6 runs a tool-use loop and emits a clear / monitor / escalate decision. Escalations surface on the doctor dashboard live via Supabase Realtime.

Users: solo vets (primary), clinic receptionists, and pet owners (Telegram only — never see the web app).

1.2 Tech Stack

Layer	Tool / Service
Web app + API	Next.js 16 (App Router) + React 19 on Vercel
Reasoning + vision	Anthropic Claude (Haiku 4.5 / Sonnet 4.6)
Speech-to-text	Deepgram nova-3
Web search tool	Tavily (cached 7 days in Postgres)
Telegram bot	grammY — polling in dev, webhook in prod
Database + Realtime + Storage	Supabase (Postgres)
Agent sidecar	Python FastAPI + LangGraph (agent/) — deferred to finals
CI	GitHub Actions (AI eval, clinic-brain-sync cron)

2. Deployment Objectives

Successful deployment means we can demo and ship without a dedicated ops engineer:

- Reproducible — every production deploy comes from a tagged commit on main; no machine-specific build steps.
- Fast — Vercel preview build under 3 min; production promotion under 5 min end-to-end.
- Secret-safe — zero secrets in the repo; every key lives in Vercel Project Settings or GitHub Repo Secrets.
- Telegram-stable — webhook delivery survives deploy (no 409 Conflict from a leftover polling process).
- Reversible — any production deploy can be rolled back in under 2 min via Vercel Promote Previous Deployment.
- Observable enough for a 3-clinic pilot — Vercel function logs + Supabase logs + GitHub Actions runs are sufficient; no APM required yet.

3. Deployment Architecture

3.1 Production Topology

The Next.js app and Python LangGraph sidecar live in the same monorepo but deploy to different platforms (Vercel + Railway/Render) because Vercel's serverless function size and runtime aren't a fit for the LangGraph triage graph.

Component	Platform	Purpose
Next.js 16 (App Router)	Vercel (serverless)	Web pages, API routes, Telegram webhook
Claude Haiku 4.5	Anthropic API	Pre-consult patient brief generation
Claude Sonnet 4.6	Anthropic API	Consult SOAP / triage tool-use loop
Deepgram nova-3	Deepgram API	Voice transcription
Tavily	Tavily API	Web search (cached 7 days)
Supabase Postgres	Supabase	Primary datastore
Supabase Realtime	Supabase	Live dashboard escalation cards
Supabase Storage	Supabase	Consult and owner photos
LangGraph sidecar	Railway / Render	Advanced triage graph (deferred)

3.2 Request Flow — Owner Message to Escalation Card

The sequence below describes the full round-trip from a Telegram owner message to a live escalation card on the doctor dashboard.

- Owner sends message to @consilium_vet_bot.
- Telegram POSTs to https://<app>/api/telegram/webhook with header X-Telegram-Bot-API-Secret-Token.
- Webhook route verifies the secret and hands the update to lib/telegram-handler.ts.
- Handler resolves the chat-id to a followups row and patient context; downloads any photo to Supabase Storage; then calls lib/llm.ts with the full conversation history.
- Claude Sonnet 4.6 runs the tool-use loop — if it emits request_photo / request_temperature, the bot asks the owner a clarifying question; if it emits emit_decision, the row is updated with decision = clear | monitor | escalate.
- Supabase Realtime pushes the row update to any open /dashboard browser tab. The escalation card appears within 1–2 seconds.

3.3 Failure Fallback

Failure	Behaviour
Claude API down	Returns structured fallback (status: agent_unavailable); Telegram sends holding reply; row left in pending for manual triage.
Tavily down or key unset	Tool registry omits tavily_search; Claude proceeds without web context.
Supabase Storage bucket missing	Photo handler falls back to inline base64 — still works, no public URL.
ANTHROPIC_API_KEY unset	App boots in mock mode; every page renders from lib/glm-fixtures.ts with no network calls.

4. Environment Configuration

All env vars are documented in `.env.local.example`. Three deployment environments exist:

Environment	Where	Trigger	Notes
Local dev	Developer laptop	<code>npm run dev + npx tsx scripts/start-bot.ts</code>	Polling bot; mock mode if no API key
Vercel Preview	Per-PR URL	Every push to a non-main branch	Telegram webhook not rewired — preview URLs do not handle bot traffic
Production	<code>consilium-tau.vercel.app</code>	Merge / push to main	Single source of truth for the live <code>@consilium_vet_bot</code> webhook

4.1 Required Environment Variables (Production)

Variable	Source	Required?
<code>NEXT_PUBLIC_APP_URL</code>	Static — <code>https://consilium-tau.vercel.app</code>	Yes
<code>ANTHROPIC_API_KEY</code>	<code>console.anthropic.com</code>	Yes — without it, app stays in mock mode
<code>DEEPGRAM_API_KEY</code>	<code>console.deepgram.com</code>	Yes — required for <code>/api/transcribe</code>
<code>TAVILY_API_KEY</code>	<code>app.tavily.com</code>	Optional — gracefully omitted if missing
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Supabase Settings > API	Yes
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Supabase Settings > API	Yes
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Supabase Settings > API	Yes — server-only, never exposed to client
<code>SUPABASE_DB_URL</code>	Supabase Settings > Database	Yes — used by <code>agent/consolidate.py</code>
<code>TELEGRAM_BOT_TOKEN</code>	<code>@BotFather</code>	Yes
<code>TELEGRAM_WEBHOOK_SECRET</code>	<code>openssl rand -hex 32</code>	Yes in production — webhook refuses to serve when unset
<code>NEXT_PUBLIC_CLINIC_ID/NAME/DOCTOR/PHONE</code>	Clinic config	Optional — defaults bake the demo identity at build time
<code>LANGGRAPH_SERVICE_URL</code>	Railway / Render	Only if Python sidecar is enabled

4.2 Per-Feature Model Overrides

Default models are baked into lib/llm.ts. Override only when a regression hits:

- ANTHROPIC_MODEL_BRIEF — default claude-haiku-4-5-20251001
- ANTHROPIC_MODEL_CONSULT — default claude-sonnet-4-6
- ANTHROPIC_MODEL_TRIAGE — default claude-sonnet-4-6

4.3 Secret Management

- Web app — Vercel Project Settings > Environment Variables, scoped to Production / Preview / Development separately. NEXT_PUBLIC_* values are inlined at build time, so changing them requires a redeploy, not just a restart.
- GitHub Actions — Repo Settings > Secrets and variables > Actions. Used by clinic-brain-sync.yml and ai-eval.yml.
- Local dev — .env.local (gitignored). Never commit.

Rotation procedure: rotate the key with the upstream provider, paste the new value in Vercel + GitHub Secrets, redeploy. Old key can stay valid for ~10 min during the swap.

5. CI/CD and Deployment Workflow

5.1 Branch Strategy

- main is the production branch. Direct pushes are allowed for hackathon velocity but every meaningful change goes via PR so the AI eval workflow runs.
- Feature branches use feat/<scope> or fix/<scope>. Vercel auto-creates preview URLs.
- No long-lived dev or staging branches — preview URLs are the staging environment.

5.2 End-to-End Deploy Flow

```
git push origin <branch>
├─┬─ GitHub Actions
│  │  └─ ai-eval.yml      (only runs if agent/** changed)
│  └─ db-migrate.yml     (only runs on push to main)
└─┬─ Vercel
   │  └─ detects Next.js, runs `next build`
   │  └─ injects env vars (Production or Preview)
   │  └─ deploys serverless functions
   └─ promotes to production URL (only on `main`)
```

5.3 Vercel Ignored-Build-Step

In Vercel Project Settings > Git > Ignored Build Step, paste the following to skip builds when only Python agent code changed:

```
git diff --quiet HEAD^ HEAD ./app ./components ./lib ./public ./supabase
package.json package-lock.json
```

5.4 AI Eval Pipeline (ai-eval.yml)

Standard unit tests do not catch prompt regressions. A 15-case eval runs against the triage agent on every PR that touches agent/**:

- 5 Escalate scenarios — severe vomiting post-op, seizures, bleeding wound, etc.
- 5 Monitor scenarios — mild lethargy, missing one meal, etc.
- 5 Clear scenarios — eating normally, back to baseline.

Each test asserts decision == expected_decision. A flipped decision fails the PR. The job has a 10-minute timeout — 15 calls finish well under that.

5.5 Database Migrations (db-migrate.yml)

Migrations live in `supabase/migrations/*.sql` (currently 0001–0009). On push to main, the workflow runs `supabase db push` against the project. Migrations are forward-only; rolling back a schema change requires a new migration.

6. Step-by-Step Deployment Process

Step 1 — Deploy Next.js to Vercel

7. Push the repo to GitHub.
8. In Vercel > New Project > Import the repo. Framework auto-detected as Next.js.
9. Build settings: leave defaults (next build, output .next).
10. Environment variables: paste every required var from Section 4.1 into the Production scope.
11. Ignored Build Step: paste the snippet from Section 5.3.
12. Click Deploy. First build takes ~2–3 min.
13. Smoke test the production URL — / should render the marketing page; /dashboard should render with seed data (if Supabase env vars are set) or fixtures (if not).

Step 2 — Provision Supabase

14. Create a Supabase project. Copy URL, anon key, service_role key, and the database connection string into Vercel + GitHub Secrets.
15. Apply migrations: supabase link --project-ref <ref> then supabase db push. Or paste each migrations/000N_*.sql in order via the SQL editor, then seed.sql.
16. Verify 0005_storage_buckets.sql ran — confirms consult-photos and owner-photos public buckets exist.
17. Verify 0002_realtime_followups.sql and 0008_realtime_patients.sql enabled Realtime on those tables.

Step 3 — Configure the Telegram Bot

18. Create the bot with @BotFather; copy the token to TELEGRAM_BOT_TOKEN (Vercel + GitHub Secrets).
19. Generate a webhook secret: openssl rand -hex 32. Set as TELEGRAM_WEBHOOK_SECRET in Vercel.
20. Stop any local polling process (pkill -f scripts/start-bot.ts). Telegram allows only one consumer per bot — running both produces 409 Conflict.
21. Register the webhook (one-time):

```
curl -X POST "https://api.telegram.org/bot<TOKEN>/setWebhook" \  
-d "url=https://consilium-tau.vercel.app/api/telegram/webhook" \  
-d "secret_token=<TELEGRAM_WEBHOOK_SECRET>"
```

22. Send /start to the bot from a test account — should reply with the chat id.
23. Seed a follow-up linked to that chat id and reply to validate the clear / monitor / escalate branches end-to-end.

Step 4 — Deploy the Python LangGraph Sidecar (Optional)

Currently deferred — the TypeScript tool-use loop in lib/llm.ts handles all flows. Only required if LANGGRAPH_SERVICE_URL is set.

24. Connect the same GitHub repo to Railway or Render.
25. Root Directory → agent.
26. Start command → `fastapi run server.py --host 0.0.0.0`.
27. Add env vars: ANTHROPIC_API_KEY, SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, CLINIC_ID.
28. Deploy. Copy the public URL into Vercel as LANGGRAPH_SERVICE_URL.
29. Smoke test: `curl <url>/health` should return `{"ok": true}`.

Step 5 — Production Validation Checklist

After every production deploy, walk through this list manually:

- ☐ / renders the marketing page with the dog mascot.
- ☐ /receptionist > Load demo data > Send to Dr. Amirah succeeds (Supabase write).
- ☐ /dashboard > realtime banner appears within 2s of the receptionist push.
- ☐ /consult > record voice (5+ sec) > transcript lands (Deepgram).
- ☐ /consult > Generate structured output > SOAP / Rx / billing / todos populate.
- ☐ Send Telegram message > bot replies; for "bleeding from incision and won't stand" the dashboard shows an escalation card.
- ☐ Open passport link from Telegram > renders, QR scannable.

If any step fails, roll back immediately. See Section 8 for rollback steps.

Step 6 — Schedulers

Clinic Brain Sync (.github/workflows/clinic-brain-sync.yml)

- Runs daily at 02:00 UTC (cron: '0 2 * * *').
- Executes agent/consolidate.py, which performs incremental consolidation by checking the updated_at watermark in the LangGraph store — only new corrections since the last sync are processed.
- Output: synthesized Clinic SOPs and Clinical Trends written back to Supabase, used as few-shot context for future Claude calls.
- Manual trigger available via GitHub > Actions > Run workflow.
- Required GitHub Secrets: SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, ANTHROPIC_API_KEY, SUPABASE_DB_URL, CLINIC_ID.

A Vercel-side cron alternative exists at `app/api/cron/consolidate-memory/route.ts`. Run ONE or the other, not both.

7. Monitoring and Error Handling

7.1 Where Logs Live

Surface	Source	Use For
Vercel > Project > Logs	Function invocation logs	API errors, latency, cold-start spikes
Vercel > Project > Deployments	Build logs	Failed builds, env-var typos
Supabase Logs	Postgres / Realtime / Storage	RLS denials, slow queries, bucket errors
GitHub > Actions	Workflow runs	AI eval failures, migration failures, cron output
Polling bot terminal (dev only)	scripts/start-bot.ts	Colour-coded boxes for owner inbound, agent reasoning, tool call, decision, outbound

7.2 Failure Handling Already in Code

- Telegram webhook — returns 200 even on internal errors so Telegram does not retry-storm; errors are logged to Vercel.
- Claude API failure — tool-use loop in lib/llm.ts catches and returns a fallback decision shape. The Telegram handler sends a holding reply to the owner.
- Deepgram timeout — /api/transcribe returns { ok: false, reason: 'stt_unavailable' }; the consult UI shows an inline error and keeps the recording so the user can retry.
- Tavily failure or rate-limit — cached lookups in tavily_cache cover the last 7 days; on miss, Claude proceeds without web context (logged at info).
- Supabase Storage missing — handler in lib/storage.ts falls back to inline base64.

7.3 Current Observability Gaps

No APM (Sentry, Datadog) — see Section 10 for future improvements. No paging / on-call rotation. Internal targets: P95 triage latency under 8s; webhook delivery success > 99%.

8. Rollback Plan

8.1 When to Roll Back

Roll back immediately if any of these are observed in production:

- Telegram webhook returns 5xx on multiple consecutive owner messages.
- /api/triage or /api/consult P95 latency exceeds 15s for 5 consecutive minutes.
- Doctor dashboard fails to load or the realtime escalation card stops appearing.
- A regression in the AI eval is discovered post-merge (e.g., a monitor case now flips to clear).
- Auth / Supabase RLS suddenly denies queries that worked previously.

8.2 Rollback Steps (Target: Under 2 Minutes)

30. Vercel > Project > Deployments > find the last known-good deployment > Promote to Production. This is instant — no rebuild.
31. If the issue was an env-var change, restore the previous value in Project Settings > Environment Variables (Vercel keeps history) and redeploy.
32. If the issue was the Telegram webhook URL, re-register against the rolled-back deployment URL using the setWebhook curl from Section 6, Step 3.
33. If the issue is a bad migration, write a forward-only fix migration — do not edit the existing migration file. Open a hotfix PR to main.
34. Post a one-line incident note in the team chat with the rolled-back commit SHA and the symptom.

8.3 What Rollback Cannot Fix

Schema migrations that already ran are not reverted by a Vercel rollback — forward-only fix migration is the only safe path.

- Telegram messages already sent to owners cannot be unsent. If a bad triage shipped, a follow-up message and a corrections-table entry are the right response.
- Supabase data writes are not transactional with the deploy. A rollback may leave rows that reference fields the older code does not know about.

9. Risks and Mitigation

Risk	Likelihood	Impact	Mitigation
Telegram webhook fails after a deploy	Medium	High	Deploy validation checklist (§6); webhook secret pre-checked in route.ts
Two bot processes (polling + webhook) collide	Medium	High	Pre-deploy: pkill -f scripts/start-bot.ts; production never runs polling script
Claude API outage or rate limit	Low	High	Fallback shape returned to user; mock-mode boots the app even with no key
Deepgram outage	Low	Medium	Voice capture fails gracefully; doctor falls back to typed notes
Tavily rate-limit	Medium	Low	7-day cache in tavily_cache; tool omitted entirely if key absent
Bad migration on main	Low	High	Forward-only fix migration; never edit applied SQL files
AI eval regression slips through	Medium	High	ai-eval.yml blocks PR merge; doctor reviews every send before transmit
Leaked SUPABASE_SERVICE_ROLE_KEY	Low	Critical	Vercel env vars are scoped server-only; rotate via Supabase + Vercel + GitHub Secrets
Webhook secret unset in prod	Low	High	Webhook route enforces this in NODE_ENV=production; surfaces immediately on first request

10. Future Improvements

These are out of scope for the hackathon submission but are the natural next steps for a real pilot:

- Sentry / OpenTelemetry — capture exceptions and traces across /api/triage and the Telegram webhook so the AI eval cases are not the only signal.
- Background queue (Inngest or a Redis worker) — move agent/consolidate.py off GitHub Actions onto an event-driven queue so consolidation can happen on every correction, not just nightly.
- Multi-tenant clinic deployment — introduce a clinic_id foreign key on every table and an RLS policy per clinic.
- Migration testing in CI — spin up a Supabase shadow DB in db-migrate.yml and apply migrations there before production push.
- Deepgram streaming — replace the batch /api/transcribe path with WebSocket streaming so doctors see transcripts word-by-word.
- Per-clinic AI eval datasets — extend the 15-case eval with the corrections logged by each pilot clinic.
- LangGraph sidecar promotion — wire LANGGRAPH_SERVICE_URL into lib/llm.ts for the triage flow and run the TS path in shadow mode to validate parity before cutover.
- Usage + cost dashboards — surface Claude / Deepgram / Tavily spend per clinic so pricing tiers can be calibrated against real unit economics.

Appendix — Quick Reference

Item	Value
Production URL	https://consilium-tau.vercel.app
Telegram Bot	@consilium_vet_bot (https://t.me/consilium_vet_bot)
Repository	https://github.com/Shawnchee/DA-Homies
Migrations	supabase/migrations/0001–0009
CI Workflows	.github/workflows/ai-eval.yml, clinic-brain-sync.yml
Env Reference	.env.local.example
Architecture Deep Dive	SAD.pdf
Demo Runbook	flow.md